

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PASHCHIMANCHAL CAMPUS

[Subject Code: EX 654]

A Minor Project Report
On

SOLAR BASED REAL TIME GREENHOUSE AUTOMATION SYSTEM

Submitted By:

ICCHYA BHATTARAI	PAS079BEI020
NANDITA TIWARI	PAS079BEI026
PRASHANT BHANDARI	PAS079BEI030
SUJAL SHRESTHA	PAS079BEI045

Submitted To:
Department of Electronics and Computer Engineering
Pashchimanchal Campus

May, 2026

LETTER OF APPROVAL

The undersigned certify that they have read and recommended to the Institute of Engineering for acceptance, a project report entitled **"Solar Based Real Time Greenhouse Automation System"** submitted by **Icchya Bhattarai, Nandita Tiwari, Prashant Bhandari, Sujal Shrestha** in partial fulfillment for the degree of Bachelor of Engineering in Electronics, Communication and Information Engineering..

Supervisor:
Asst. Professor Er. Hom Nath Tiwari
Department of Electronics and Computer Engineering

Er. Sanjita Lamichhane
External Examiner

Asst. Professor Er. Hom Nath Tiwari
Head of Department, Department of Electronics and Computer Engineering

DATE OF APPROVAL: May, 2026

ACKNOWLEDGEMENT

First and foremost, we would like to express our sincere gratitude to our respected supervisor, **Assistant Professor Er. Hom Nath Tiwari**, for his invaluable guidance, continuous support, and encouragement throughout the course of this project. His insightful feedback and motivation were instrumental in shaping our understanding and execution of the project work.

We are equally thankful to the **Department of Electronics and Computer Engineering**, **Pashchimanchal Campus**, for providing the necessary resources, environment, and platform to carry out our minor project titled **"Solar Based Real Time Green- house Automation System"**.

We extend our heartfelt thanks to the **Head of Department, Assistant Professor Er. Hom Nath Tiwari** , for his administrative support and encouragement. We would also like to acknowledge the contribution of the **Deputy Head of Department, Er. Ramesh Thapa**, for his guidance and facilitation during the project period.

Last but not least, we would also like to thank our friends and colleagues for their meaningful discussions, brainstorming sessions, and moral support. Your input has enriched the quality of this report and helped us navigate through challenging concepts.

Icchya Bhattarai
Nandita Tiwari
Prashant Bhandari
Sujal Shrestha

ABSTRACT

Agriculture is the backbone of Nepal's economy. Farming in Nepal relies heavily on manual labour for irrigation, humidity and temperature regulation making it vulnerable to human error, inconsistent monitoring and delayed environmental response. This project presents a solar based, real-time greenhouse automation system built around an ESP32 microcontroller that continuously monitors temperature, humidity, and soil moisture, automatically activates a water pump for irrigation and a mist maker for humidity and temperature regulation whenever conditions cross predefined thresholds for the cowpeas. The conitionus monitoring components are powered entirely off-grid through a solar panel and lithium polymer battery, while sensor data is displayed locally on an OLED screen and transmitted wirelessly via BLE to a web dashboard for real-time monitoring having auto and manual control. Experimental results confirmed that the environmental parameters were more stable in AUTO mode than during manual mode. The continuous operations consumed about 0.64 watts during steady-state operation with over 85% battery voltage monitoring accuracy. BLE connectivity remained stable at -53 dBm RSSI with a 43.51 ms connection interval across 50–60 m. The solar charging was efficient at maximum 91 mA. These results collectively validate that the automated greenhouse system provides greater environmental stability than manual greenhouse management and offers a deployable renewable energy integrated foundation for precision agriculture in Nepal.

Keywords : *BLE, ESP-32, Greenhouse Automation , Real-Time Monitoring , Solar-Powered*

TABLE OF CONTENTS

CERTIFICATE OF APPROVAL i

ACKNOWLEDGEMENT ii

ABSTRACT iii

LIST OF FIGURES vi

LIST OF TABLES vii

LIST OF ABBREVIATIONS viii

1 INTRODUCTION 1

1.1 Background 1

1.2 Problem statement 2

1.3 Objectives 2

2 LITERATURE REVIEW 3

3 REQUIREMENTS 5

3.1 Hardware Requirements 5

3.2 Software Requirements 11

4 METHODOLOGY 13

4.1 Block Diagram 13

4.2 Flow Chart 15

4.3 Hardware Setup and System Design 16

4.3.1 Sensor Node (Data Acquisition Unit) 16

4.3.2 Control Unit 16

4.3.3 Power Management Unit (Solar Charging System) 18

4.3.4 Power Budget Analysis 18

4.3.5 Experimental Power and Energy Calculation (Measured Data) 19

4.3.6 Battery Voltage Monitoring and Calibration 21

4.3.7 Physical Arrangement of the System 26

4.3.8 Pin Configuration and Peripheral Interface 28

4.3.9 Data Representation 29

4.4 Automation Logic and Threshold Calibration 30

4.5 Sensor Data Filtering and Noise Reduction 32

4.6 BLE-Based Monitoring and Data Management System 35
4.6.1 BLE Communication Architecture 35
4.6.2 User Monitoring Interface 35
4.6.3 Data Handling and Logging 36
4.6.4 System Data Flow Architecture 36
4.6.5 Monitoring Capability 36
4.6.6 Performance Analysis 37
4.7 Software System Design 38
4.7.1 System Architecture and Operation 38
4.7.2 Firmware and BLE Communication 38
4.7.3 Backend, Frontend, and System Flow 39
4.7.4 Database, API and Deployment 41

5 EPILOGUE **43**
5.1 Results and Findings 43
5.1.1 Relation in Auto and Manual Mode 43
5.1.2 Environmental Time Series 45
5.1.3 Solar Charging and Energy Performance Analysis 47
5.1.4 Key Findings 49
5.2 Limitations and Future Enhancements 49
5.2.1 Limitations 49
5.2.2 Future Enhancements 50
5.3 Conclusion 50

REFERENCES **51**

LIST OF FIGURES

3.1 Solar Panel 5
3.2 Li-Po 3.7V Pouch Battery 6
3.3 ESP32 Board 6
3.4 DHT11 Sensor 7
3.5 Soil Moisture Sensor 8
3.6 DC Water Pump 8
3.7 Ultrasonic Mist Maker 9
3.8 OLED Display Board 9
3.9 TP4056 Charging Module 10
3.10 MP1584 Voltage Regulator 10
3.11 IRLZ44N MOSFET 11
4.1 Block Diagram 13
4.2 Flow Chart of Complete Workflow 15
4.3 Schematic of Voltage Regulator 17
4.4 DC Pump Circuit 17
4.5 Mist Maker Circuit 17
4.6 Schematic of Voltage Divider 21
4.7 Code of Battery Monitoring 26
4.8 Physical Arrangement of Greenhouse 27
4.9 ESP32 Pin Configuration 29
4.10 Code of ESP32 Actuator Control Logic 31
4.11 Code of Threshold Values 31
4.12 Code of DHT11 Moving Average Filtering Algorithm 34
4.13 RSSI Chart 37
4.14 Code of BLE UUID Configuration for ESP32 Communication 39
4.15 Frontend Dashboard 40
5.1 Correlation analysis in AUTO Mode 43
5.2 Correlation analysis in MANUAL Mode 44
5.3 Environmental Time Series 45
5.4 Activation Regions of Mist Maker 46
5.5 Motor Pump ON/OFF 47
5.6 Solar Current vs Charging Current 47
5.7 Battery Voltage Variation 48
5.8 Solar Panel Voltage Variation 49

LIST OF TABLES

4.1 System Power Consumption Table 19
4.2 ESP32 Pin Configuration and Peripheral Interface 28
5.1 Performance Comparison Between AUTO and MANUAL Operating Modes 45

LIST OF ABBREVIATIONS

ADC	Analog to Digital Converter
AI	Artificial Intelligence
BLE	Bluetooth Low Energy
BMS	Battery Management System
CC-CV	Constant Current–Constant Voltage
CSS	Cascading Style Sheets
DC	Direct Current
DHT	Digital Humidity and Temperature Sensor
GATT	Generic Attribute Profile
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit
IoT	Internet of Things
LCD	Liquid Crystal Display
LiPo	Lithium Polymer
LoRa	Long Range
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
MPPT	Maximum Power Point Tracking
nRF	Nordic Radio Frequency
NTC	Negative Temperature Coefficient
OLED	Organic Light Emitting Diode
ORM	Object Relational Mapping
PCB	Printed Circuit Board
PWM	Pulse Width Modulation
REST API	Representational State Transfer Application Programming Interface
SCL	Serial Clock
SDA	Serial Data Line
SoC	System on a Chip
SQL	Structured Query Language
UUID	Universally Unique Identifier

1. INTRODUCTION

1.1. Background

Nepal is an agricultural country. Here, the majority of people are dependent on agriculture for their livelihood. But agricultural productivity here is comparatively less due to factors like complex geographical conditions, limited resources and facilities, and increasing climate variations. Many farmers still rely on seasonal crops that are affected by temperature changes, irregular rainfall, and extreme weather. These challenges are more in rural and mountainous regions, where access to modern agricultural methods and reliable energy sources is limited. Lately, greenhouse farming is emerging as a solution to all these issues. However, in Nepal, greenhouse management is still performed manually, requiring continuous human involvement to monitor temperature, humidity, soil moisture, and other parameters. This is time consuming and not so accurate, especially for those farmers who may not be able to constantly supervise the greenhouse throughout the day.

Recently with the advancements in automation and related technologies, it is possible to monitor and control greenhouse environments in real time. But, most existing automated systems depend on electricity, which is usually unreliable or unavailable in many rural areas of Nepal. Power shortages and inconsistent supply further limit the scope of such technologies.

Nepal has significant potential for solar energy so integrating the solar power with greenhouse automation offers a sustainable and reliable solution for remote and off-grid agricultural settings. A solar based real time greenhouse automation system can ensure continuous operation without depending on the conventional power sources.

This project focuses on the design and implementation of a solar powered, real time greenhouse automation system that monitors key environmental parameters of a greenhouse i.e, temperature, humidity and soil moisture. The system will automatically control the mentioned greenhouse conditions using an ESP32 and display the real time data in the greenhouse as well as dashboard. By combining the system with solar energy, the proposed system targets to improve agricultural productivity, reduce manual labor, and support sustainable farming practices suitable for Nepal.

1.2. Problem statement

Traditional greenhouse farming in Nepal is majorly based in manual labour for temperature regulation and irrigation, which is physically demanding and highly susceptible to error. Farmers in Nepal's diverse topographical regions lack the real-time data and automated tools necessary to respond to sudden environmental changes. Although automated greenhouse systems exist, they are often costly, complicated, and depend on a continuous power supply, which is unreliable in many rural and semi rural areas. While solar power offers a sustainable solution for these remote backgrounds, the main challenge becomes the lack of integrated systems that can independently monitor and adjust the internal parameters. There is a need for a low cost automation that autonomously regulates temperature, humidity, and soil moisture. By using solar power to control the actuator responses, the system would eliminate the constant manual supervision, stabilize crop yields, and modernize agriculture in areas where technical resources can be limited.

1.3. Objectives

1. To develop an automated, real-time greenhouse monitoring system that improves operational efficiency.
2. To develop a cost-effective, solar-powered system for efficient and sustainable operation.

2. LITERATURE REVIEW

Candy et al. [1] investigated the design, construction and thermal performance of typical solar greenhouses in Humla, Nepal, which demonstrates the effect of the high altitude conditions and limited resources on crop production. Their work shows that insulated walls, double-layer glazing and thermal mass are effective at keeping air and soil temperatures suitable for winter crops. This study guided the decisions about the structure and materials to build the greenhouse and showed the potential to increase the crop yields in cold climates. But the system was based on manual ventilation only and did not have any real time monitoring and automation features which limited its efficiency and responsiveness. The limitations are solved by the proposed system using IoT based real time monitoring and automated climate control.

Siddiqui et al. [2] developed a smart greenhouse system where he used sensors and a micro-controller to monitor temperature, humidity, soil moisture and light intensity which is automatically adjusted using actuators in environmental conditions. Web and mobile applications are used to enable monitoring, enabling farmers to control greenhouse conditions without having to be physically present. Their approach shows optimisation of microclimatic conditions, crop yield and reduction of manual labour with IoT based automation and is suitable for small scale farmers.

Abou-Mehdi-Hassani et al. [3] presents a systematic review on the application of smart greenhouse monitoring systems. The review focuses on the application of Internet of Things technology in the collection of live data and automatic climate control. From the 54 articles produced from 2019 up to 2023, it is detected that there is improved resource management and increased crop production when IoT is combined with AI, ML and fuzzy logic. The researchers have also underscored the need for energy-efficient and solar-powered greenhouse systems.

Omid and Shafaei [4] examines temperature and relative humidity variations inside greenhouses compared to external conditions using multiple sensors. Their output showed that temperatures inside the greenhouse are higher due to solar radiation which is trapped inside and vertical temperature gradients exist within the structure. They used fans and heaters for automated control that can maintain stable conditions in greenhouses demonstrating the importance of continuous environmental monitoring for effective greenhouse management.

Ayuba [5] did a project where he used a system to measure things using the DHT11 sensor. He tested to see how the DHT11 sensor could check the humidity and tem-

perature. The DHT11 sensor is special because it has a thermistor and a capacitive humidity element that work together to give a signal to a microcontroller. This signal is sent through one wire. This makes it easy to design a system. The DHT11 sensor is a choice because it is cheap and it can give you the information you need in real time. This is useful, for automated systems that need to check the environment. The DHT11 sensor is a low-cost sensor that can do this job well.

Singh [6] evaluated the application of solar photovoltaic systems in the powering of automated agricultural infrastructure. The study explains how solar energy is converted into electrical energy using the photovoltaic effect and highlights its use in the powering of sensors, irrigation pumps and control systems. The research concludes that solar-based systems provide a sustainable and reliable energy source for remote agricultural applications, reducing dependency on conventional grid electricity while ensuring continuous operation of automated systems.

3. REQUIREMENTS

The various tools used for the implementation of this project are listed below:

3.1. Hardware Requirements

1. **Solar Panel**

   Thin-film solar cells are a type of solar cell made by depositing thin layers of photovoltaic material onto a glass, plastic or metal substrate. Polycrystalline silicon based solar cells show energy conversion efficiencies of 8% and high manufacturing yields of $> 90\%$. [7]

   A solar panel converts sunlight into DC electricity using the photovoltaic effect. When sunlight falls on the panel, it releases electrons and produces an electric current.

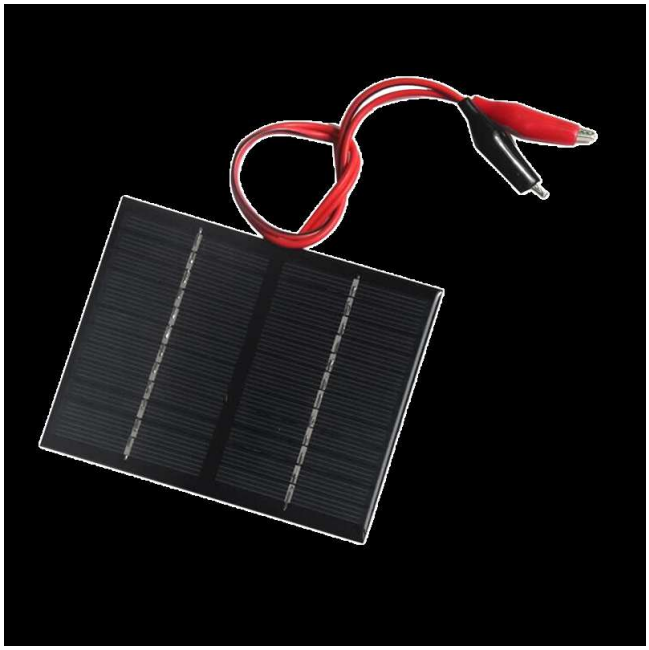


Figure 3.1: Solar Panel

2. **Li-Po Battery**

   LiPo batteries, short for lithium polymer batteries, are a special type of lithium-ion batteries that offer improved performance and flexibility. These batteries use a solid or gel-like polymer electrolyte instead of the liquid electrolytes found in conventional lithium-ion batteries. A LiPo battery consists of several main components: an anode, a cathode, a polymer electrolyte, and a

separator. [8]

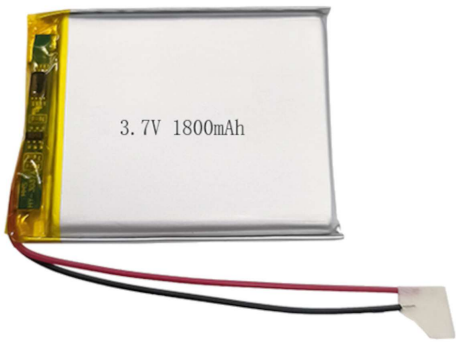


Figure 3.2: Li-Po 3.7V Pouch Battery

3. **ESP32**

   ESP32 is a series of low-cost and low-power System on a Chip (SoC) microcontrollers developed by Espressif that include Wi-Fi and Bluetooth wireless capabilities and dual-core processor. [9]

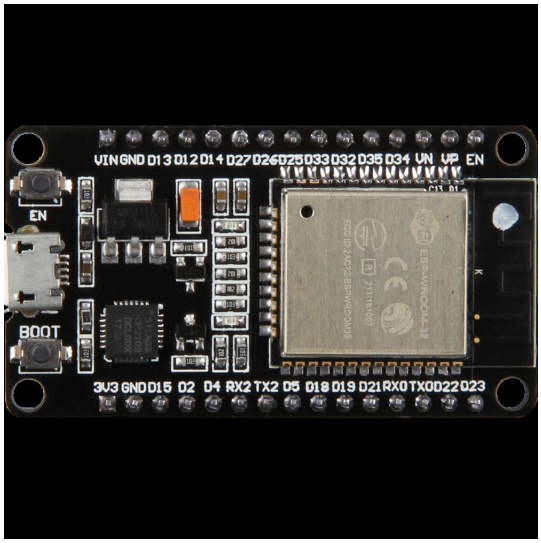


Figure 3.3: ESP32 Board

4. **DHT11 Sensor**

   DHT11 Temperature & Humidity Sensor features a temperature & humidity sensor complex with a calibrated digital signal output. This sensor includes

a resistive-type humidity measurement component and an NTC temperature measurement component, and connects to a high performance 8-bit microcontroller. The DHT11 sensor provides relative humidity value in percentage (20% to 90% RH) and temperature values in degree Celsius (0 to 50 °C). The sensor provides a humidity measurement accuracy of ±5% RH and a temperature accuracy of ±2 °C [10]

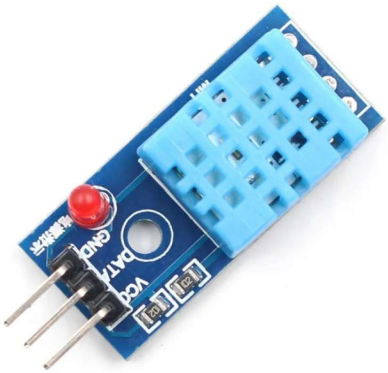


Figure 3.4: DHT11 Sensor

5. **Soil Moisture Sensor**

   The soil moisture sensor or the hygrometer is usually used to detect the humidity of the soil. So, it is perfect to build an automatic watering system or to monitor the soil moisture of your plants. The sensor is set up by two pieces: the electronic board and the sensor probe with two pads, that detects the water content. The output can be an analog signal and so get a value between 0 and 4096. [11]

   It works based on changes in the soil's electrical properties (either resistance or capacitance). In resistive sensors, higher moisture reduces electrical resistance between the probes due to increased conductivity. In capacitive sensors, the dielectric constant of the soil changes with water content, altering the capacitance, which is then converted into a measurable signal.

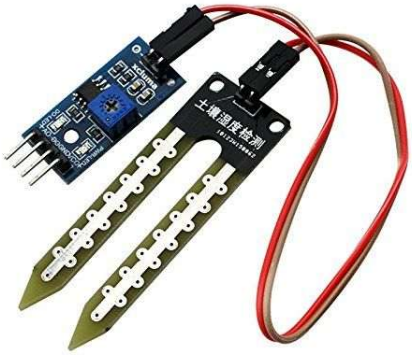


Figure 3.5: Soil Moisture Sensor

6. **DC Water Pump**

   A water pump is a mechanical device designed to move water from one location to another, utilizing pressure changes within a system. The pump's motor, which can be electric (AC or DC) or gas-powered, drives this process. Water enters through an inlet valve or pipe, moves through the impeller chamber, and exits through an outlet valve or pipe. [12]



Figure 3.6: DC Water Pump

7. **Mist Maker**

   The ultrasonic mist fogger, also known as ultrasonic mister/fogger, is a relatively small device which is commonly placed in a container filled with water,

and the high-frequency ultrasonic oscillator can break water into countless water droplets which are sprayed into air as a fog-like cloud. [13]



Figure 3.7: Ultrasonic Mist Maker

8. **OLED Display**

   OLED (Organic Light Emitting Diodes) is a flat light emitting technology, made by placing a series of organic thin films between two conductors. When electrical current is applied, a bright light is emitted. OLEDs are emissive displays that do not require a backlight and so are thinner and more efficient than LCD displays. [14]

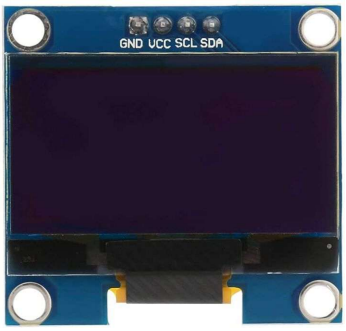


Figure 3.8: OLED Display Board

9. **TP4056**

   The TP4056 is a complete constant-current/constant-voltage linear charger for single cell lithium-ion batteries. It is suitable for portable applications

and automatically terminates charging when current drops to 1/10th of programmed value. [15]

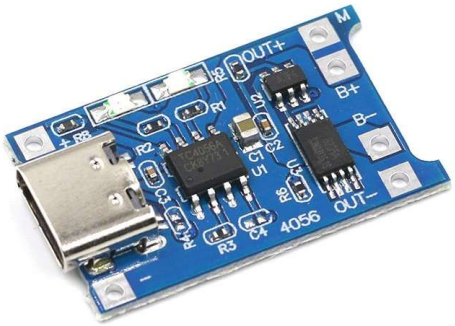


Figure 3.9: TP4056 Charging Module

10. **MP1584**

The MP1584 is a high frequency step-down switching regulator with integrated MOSFET providing 3A output capability and efficient power conversion. [16]



Figure 3.10: MP1584 Voltage Regulator

11. **IRLZ44N MOSFET**

The IRLZ44N is an N-channel MOSFET with low resistance and fast switching capability. It supports high current applications with gate threshold voltage of 1–2V.[17]



Figure 3.11: IRLZ44N MOSFET

3.2. Software Requirements

1. **Platform IO**

   PlatformIO is a cross-platform, cross-architecture, multiple framework, professional tool for embedded systems engineers and for software developers who write applications for embedded products. [18]

   It handles firmware development, library management, and uploading code to the ESP32.

2. **KiCad**

   KiCad is a free and open-source electronics design automation (EDA) suite. It features schematic capture, integrated circuit simulation, printed circuit board (PCB) layout, 3D rendering, and plotting/data export to numerous formats. KiCad also includes a high-quality component library featuring thousands of symbols, footprints, and 3D models. [19]

3. **Bluetooth Low Energy (BLE)**

   Bluetooth Low Energy is a communication protocol. It acts as a rulebook that devices must adhere to in order to communicate or exchange information. The Bluetooth protocol stack is broadly broken down into three primary components or subsystems i.e. application, host and controller. Inside each of these blocks are distinct layers. [20]

BLE can cover up to 100 meters, but the exact distance varies depending on the environmental noise and implementation.[21] It is preferred as it offers lower power consumption and is well-suited for our required setup. Data packets containing humidity, temperature and moisture data are sent at regular intervals to the backend and data is stored.

4. **Backend System**

   The backend is implemented using Node.js with NestJS framework which provides modular architecture for REST API services. It handles data ingestion from the BLE gateway, validates incoming sensor data, manages system logic and coordinates actuator control through command handling.

   The backend also supports alert generation during extreme conditions, system state management, and communication with the database layer. Structured APIs enable interaction with the frontend dashboard, while internal services ensure reliable processing and routing of sensor data and control commands.

5. **Frontend System**

   The user interface is developed using Next.js with React and TypeScript to build a responsive dashboard for monitoring and control.

   The interface displays environmental parameters temperature, humidity, soil moisture and actuator states through dynamic components. Styling is handled using Tailwind CSS, while Recharts is used for visualizing historical and real-time data trends.

   Efficient data fetching, caching, and synchronization are managed using TanStack Query, enabling near updates without requiring continuous page reloads.

6. **Database System**

   The system uses PostgreSQL as relational database. It maintains structured records of sensor readings, actuator states, user information, alerts and system configurations.

   Database interaction is handled through Prisma ORM, which provides type-safe queries, schema management, and efficient data access.

4. METHODOLOGY

4.1. Block Diagram

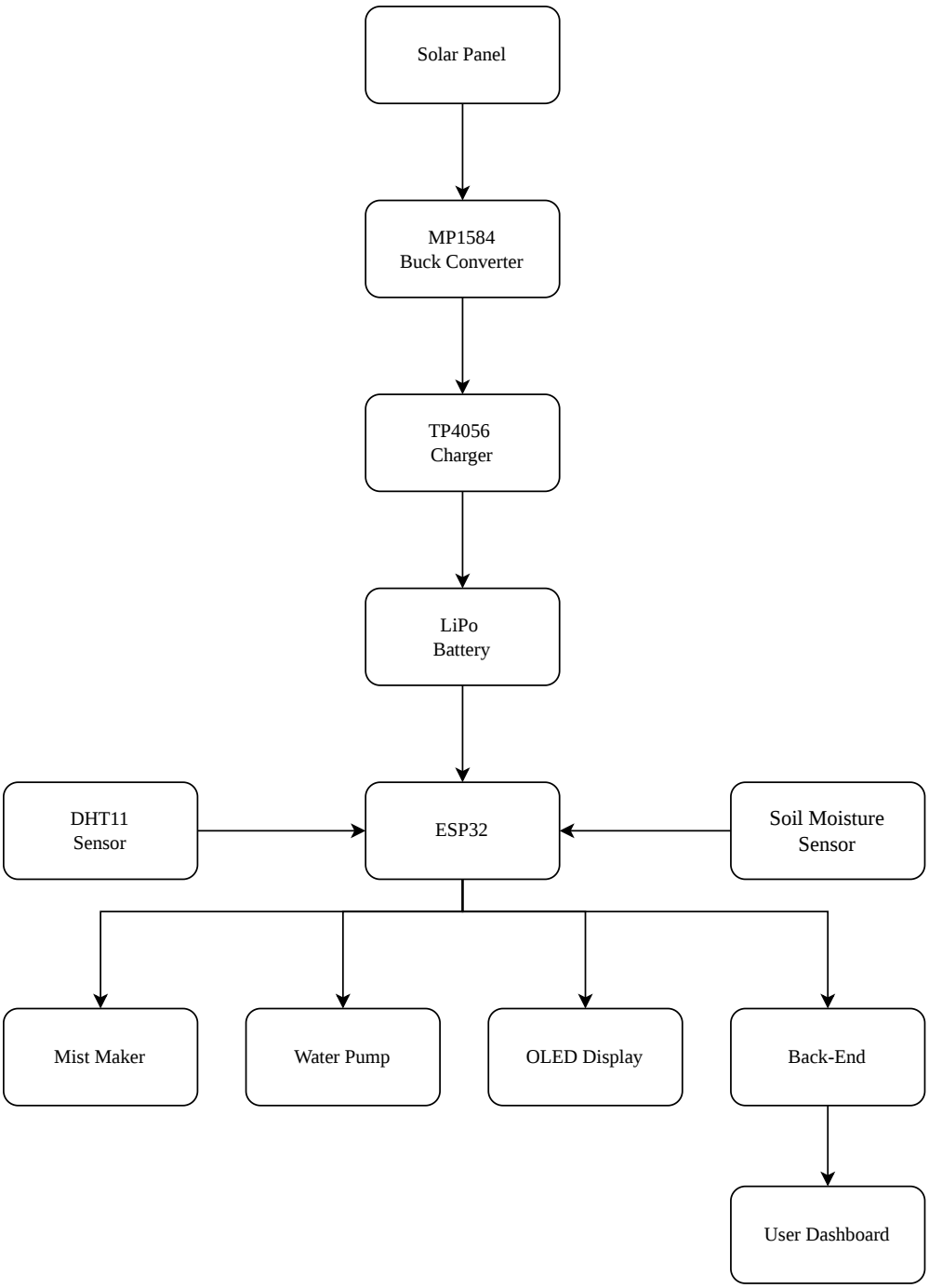


Figure 4.1: Block Diagram

The figure illustrates the architecture and working principle of the greenhouse monitoring and automation. The system is designed to provide automated environmental control, monitoring and efficient power management.

The system consists of a solar panel that converts solar energy into electrical energy. The generated power is regulated by an MP1584 buck converter, then the TP4056 charging module charges the single cell LiPo battery. The LiPo battery acts as energy storage unit.

The ESP32 microcontroller serves as the control unit of the system. It interfaces with environmental sensors, actuators, display modules and communication protocol. The DHT11 sensor measures the humidity and temperature inside the greenhouse while soil moisture sensor measures the soil moisture level. These sensors transmit data to ESP32 for processing.

Based on sensor readings, microcontroller automatically controls the actuators. When the humidity level falls below the pre-defined threshold value, ESP32 activates humidifier to maintain humidity conditions. In addition, when the temperature levels rise to threshold, the mist maker is still on to regulate the temperature unless the humidity values are exceeded. Similarly, when soil moisture level becomes insufficient, the controller turns on the water pump to irrigate the plants.

The processed environmental data is displayed locally on the OLED display allowing users to observe temperature, humidity and soil moisture values.

For remote monitoring, the ESP32 transmits the collected data to the back-end system through BLE protocol. The back-end database stores and manages the sensor data, which is then accessed through the user dashboard interface. This enables users to remotely monitor greenhouse conditions, observe and control the operational status.

4.2. Flow Chart

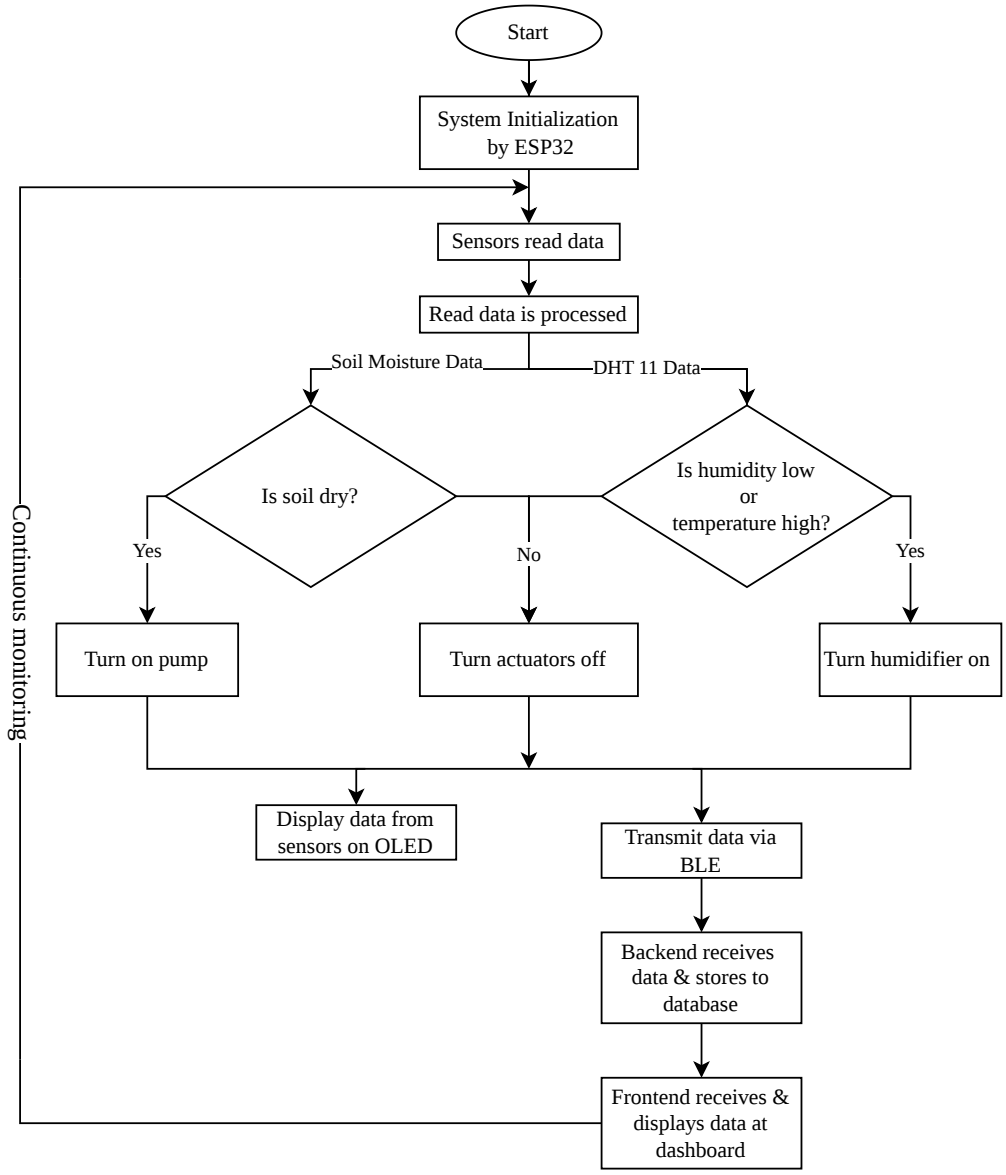


Figure 4.2: Flow Chart of Complete Workflow

4.3. Hardware Setup and System Design

The system is built using a single ESP32 microcontroller that performs environmental data acquisition, processing, decision making, and actuator control.

4.3.1. Sensor Node (Data Acquisition Unit)

The sensor node collects the environmental parameters of the greenhouse and transmits them to the control unit.

- **Function:** Reads sensor data and transmits it
- **Sensors Used:**
  - DHT11 sensor for humidity and temperature values
  - Soil moisture sensor for soil moisture levels
- **Power Source:** Solar-powered battery system
- **Operation Mode:** Periodic sampling to minimize power consumption

4.3.2. Control Unit

The control unit acts as the central processing system, receiving sensor data and controlling actuators based on predefined logic.

- **Controller:** ESP32 microcontroller
- **Function:** Processes incoming data and executes control decisions
- **Data Handling:** Compares real-time values with predefined thresholds
- **Actuators Controlled:**
  - Water pump for irrigation
  - Mist Maker for temperature and humidity regulation
- **Output Control:** Each actuator is driven through a dedicated GPIO pin configured as a digital output. The control signals are either ON/OFF or PWM-based depending on the required operation. IRLZ44N MOSFETs are used for switching high-power devices safely.A 1N4007 diode is connected across the 5V DC pump as a flyback diode to suppress back EMF generated during switching to protect the IRLZ44N MOSFET from voltage spikes and ensure reliable operation of the motor control circuit.

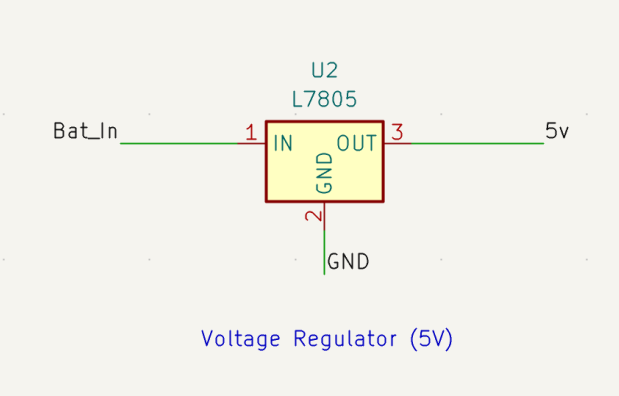


Figure 4.3: Schematic of Voltage Regulator

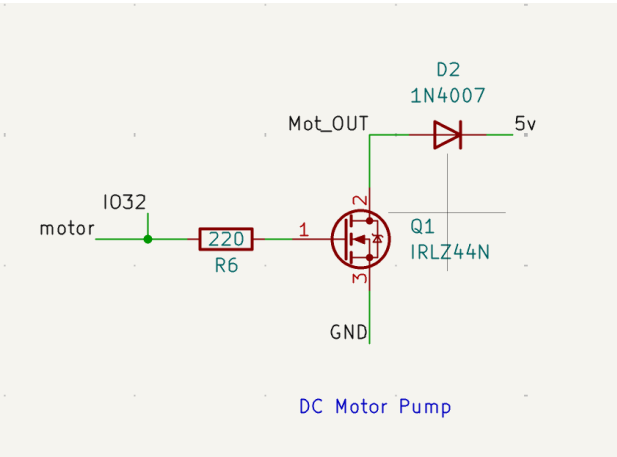


Figure 4.4: DC Pump Circuit

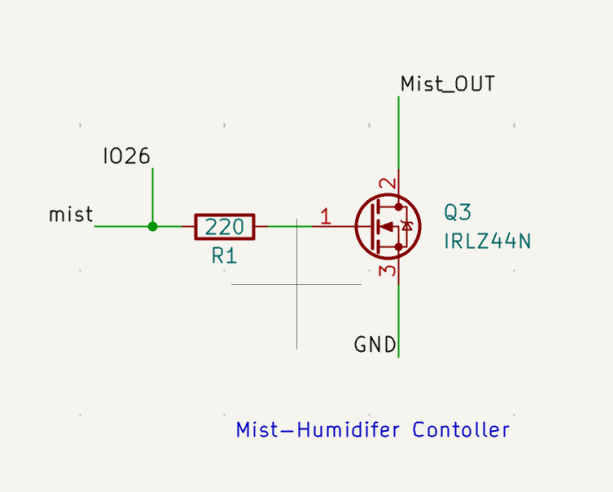


Figure 4.5: Mist Maker Circuit

4.3.3. Power Management Unit (Solar Charging System)

The power management unit is responsible for energy generation, storage, regulation, and distribution within the system. It ensures continuous operation of the greenhouse automation system using solar energy.

- **Solar Panel (Energy Generation Unit)** : The solar panel acts as the primary energy source of the system. It converts solar energy into DC electrical energy, which is used to charge the battery through the charging circuit. The panel is positioned at an optimal angle to maximize sunlight exposure and improve efficiency.
- **MP1584 Buck Converter**: The MP1584 DC-DC buck converter is used for voltage regulation and step-down conversion. It steps down the solar panel voltage to stable levels required by the TP4056. It ensures consistent voltage supply and improves system power efficiency.
- **Charging Module- TP4056** : The TP4056 module is responsible for charging of single cell LiPo battery. It operates using a Constant Current–Constant Voltage (CC–CV) charging algorithm which ensures controlled charging current prevents overcharging to protect the battery.
- **LiPo Battery** : The 3.7V LiPo battery as the energy storage unit of the system. It stores the energy generated by the solar panel and supplies power to the system during no sunlight. It acts as a power backup and ensures uninterrupted operation of the greenhouse monitoring.

4.3.4. Power Budget Analysis

The total power consumption of the system is estimated by summing individual component loads under normal operating conditions as per their available datasheets.

S.N.	Component	Voltage (V)	Current (A)	Power (W)
1	DC Water Pump	5	0.2	1.0
2	Mist Maker	5	0.3	1.5
3	ESP32 Controller	3.3	0.13	0.429
4	OLED Display	3.3	0.012	0.04
5	Soil Moisture Sensor	3.3	0.035	0.12
6	DHT11 Sensor	3.3	0.002	0.0066
7	MP1584 (Power management)	4.5 - 28	0.1	0.5
8	TP4056	5	0.05	0.25

Table 4.1: System Power Consumption Table

The DC water pump and mist maker are intermittent loads that operate only occasionally when triggered by environmental conditions. Therefore, they are not included in the continuous power consumption calculation.

Total Continuous Power Consumption:

$$P = 0.429 + 0.04 + 0.12 + 0.0066 = 0.59W$$

4.3.5. Experimental Power and Energy Calculation (Measured Data)

Battery configuration: 2×3.7 V, 1800 mAh in series

Voltage = 7.16 V Capacity = 1.8 Ah

Measured Operating Conditions:

Battery voltage: V = 7.4 V

Initial current (startup condition): $I_1 = 0.11A$

Steady-state current (normal operation): $I_2 = 0.09A$

Startup Power (Transient Condition):

$$P_1 = V \times I_1 = 7.16 \times 0.11 = 0.79W$$

Steady-State Power (Normal Operation):

$$P_2 = V \times I_2 = 7.16 \times 0.09 = 0.64W$$

For energy estimation, steady-state power is considered:

$$P_{avg} = 0.64W$$

Battery energy:

$$E_{battery} = V \times Ah = 7.4 \times 1.8 = 13.32Wh$$

Runtime calculation:

$$\text{Runtime} = \frac{E_{battery}}{P}$$

$$\text{Runtime} = \frac{13.32}{0.64} = 20.8 \text{ hours}$$

Including efficiency factor:

$$\text{Runtime} = \frac{\eta E_{battery}}{P}$$

$$\text{Runtime} = \frac{0.85 \times 13.32}{0.64}$$

The factor $\eta = 0.85$ is used to represent practical system efficiency. In real operation, not all the stored battery energy is delivered to the load due to losses in voltage regulation, internal resistance of the battery and transient current spikes.

$$\text{Runtime} = \frac{11.32}{0.64} = 17.7 \text{ hours}$$

Experimental Validation

A real-time evaluation was conducted over a duration of 1.5 hours. During this period, the battery voltage decreased from 7.4 V to 7.2 V under normal system operation.

Voltage drop rate:

$$\frac{7.4 - 7.2}{1.25} = 0.16V/hour$$

LiPo battery operating range:

- Full charge voltage: 8.4 V
- Cutoff voltage: 6.0 V
- Usable voltage range:

$$8.4 - 6.0 = 2.4V$$

Estimated runtime:

$$t = \frac{2.4}{0.16}$$

$$t = 15 \text{ hours}$$

The experimental results show that the system consumes higher power (0.64 W) in steady-state operation compared to the theoretical full-load condition (0.59 W). Based on the steady-state power consumption, the system achieves an estimated runtime of approximately 15 hours on a single battery cycle.

However, the experimentally estimated runtime was found to be approximately 15 hours during real-time operation. The deviation of approximately 15% is due to actuator switching loads and real-world operating variations.

4.3.6. Battery Voltage Monitoring and Calibration

To monitor the battery voltage using the ESP32 ADC, a voltage divider circuit was implemented to scale the battery voltage into the measurable range of the ESP32 ADC (0 V – 3.3 V).

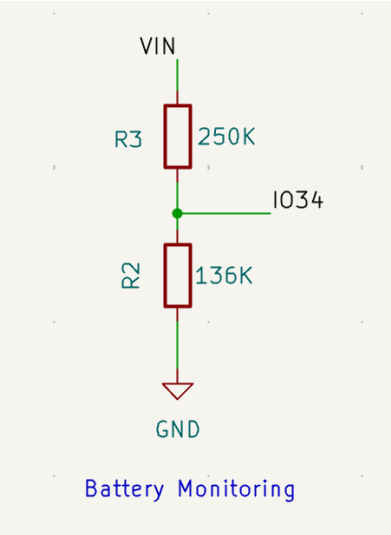


Figure 4.6: Schematic of Voltage Divider

The resistor divider consists of:

$$R_3 = 250 k\Omega$$

$$R_2 = 136k\Omega$$

The output voltage of the divider is given by:

$$V_{out} = \frac{R_2}{R_2 + R_3} \cdot V_{in}$$

Substituting the resistor values:

$$V_{out} = \frac{136}{136 + 250} \cdot V_{in}$$

$$V_{out} = \frac{136}{386} \cdot V_{in}$$

$$V_{out} = 0.3523 \cdot V_{in}$$

For a maximum battery voltage of:

$$V_{in} = 8.4V$$

The divider output becomes:

$$V_{out} = 0.3523 \times 8.4$$

$$V_{out} = 2.96V$$

which is safely within the ESP32 ADC input range.

ADC Conversion Model

The ESP32 ADC converts the analog voltage into a 12-bit digital value ranging from 0 to 4095.

The ADC conversion equation is:

$$ADC = \frac{V_{out}}{V_{ref}} \times 4095$$

where:

$$V_{ref} = 3.3V$$

Rearranging:

$$V_{out} = \frac{ADC}{4095} \times 3.3$$

Substituting the voltage divider relationship:

$$\frac{ADC}{4095} \times 3.3 = 0.3523 \times V_{in}$$

Solving for V_{in}:

$$V_{in} = \frac{ADC}{4095} \times \frac{3.3}{0.3523}$$

Thus, the battery voltage can be expressed as:

$$V_{in} = ADC \times k_{\text{theoretical}}$$

where:

$$k_{\text{theoretical}} = \frac{3.3}{4095} \times \frac{R_2 + R_3}{R_2}$$

Substituting the resistor values:

$$k_{\text{theoretical}} = \frac{3.3}{4095} \times \frac{386}{136}$$

$$k_{\text{theoretical}} = 0.00230$$

Experimental Calibration

To improve measurement accuracy, a single-point experimental calibration was performed.

Measured values:

$$V_{in} = 7.37V$$

$$ADC = 2965$$

The experimental calibration constant is:

$$k_{\text{exp}} = \frac{V_{in}}{ADC}$$

$$k_{\text{exp}} = \frac{7.37}{2965}$$

$$k_{\text{exp}} = 0.0024898649$$

This experimentally obtained constant was used in the firmware for battery voltage estimation.

Practical Measurement Observation

During experimental testing, the actual battery voltage measured using a multimeter was:

$$V_{\text{actual}} = 7.37V$$

The voltage measured using the ESP32 ADC fluctuated within the range:

$$V_{\text{measured}} = 7.37V \text{ to } 7.40V$$

The fluctuation was mainly caused by resistor tolerance, ADC noise, reference voltage variation, and minor electrical disturbances in the circuit.

To reduce measurement noise and improve accuracy, multiple ADC samples were taken and averaged in software. A total of:

$$NUM_SAMPLES = 2500$$

samples were collected for each reading, and their average value was used for the final voltage estimation.

The averaging process significantly improved the measurement stability and reduced random fluctuations in the ADC readings.

Measurement Accuracy and Efficiency

The practical voltage measurement error was very small:

$$E_{\text{voltage}} = |V_{\text{measured}} - V_{\text{actual}}|$$

Considering the maximum observed value:

$$E_{\text{voltage}} = |7.40 - 7.37|$$

$$E_{\text{voltage}} = 0.03V$$

The percentage error is:

$$E_{\%} = \frac{|7.40 - 7.37|}{7.37} \times 100$$

$$E_{\%} = 0.41\%$$

Thus, the practical measurement efficiency becomes:

$$\eta = (100 - 0.41)\%$$

$$\eta = 99.59\%$$

On another measurement instance, the input voltage was found to be:

$$V_{in} = 7.20V$$

Using the same calibrated system, the measurement was evaluated and the efficiency was found to be:

$$\eta = 97.3\%$$

This shows that variations in input voltage can occur during different instances, but the system holds an acceptable accuracy.

Therefore, the implemented battery voltage monitoring system achieved measurement accuracy of approximately above 95% under practical operating conditions.

It should be noted that the above measurement efficiency was obtained under normal monitoring operation of the system, where the ESP32, DHT11 sensor, soil moisture sensor, and OLED display were active. Under these conditions, the system consumed relatively low power, resulting in highly stable voltage measurements and accuracy above 95%.

However, during actuator operation, additional components such as the mist maker and water pump were activated. These actuators introduced higher current demand and increased the total power consumption of the system to approximately 3 W under peak load conditions. The sudden increase in current drawn causes battery discharge and temporary voltage drops which reduced the measurement accuracy to about 80%.

Since the actuators are not continuously active, the core system remains unaffected by them during normal operation. These actuator modules can also be powered using separate power sources if required. Hence, the above 95% measurement accuracy observed during regular monitoring conditions remains valid for the sensing and battery monitoring operations of the system.

Implementation

```
#define VOLTAGE_STEP 0.0024898648648649
#define NUM_SAMPLES 2500
float readBatteryVoltage()
{
    unsigned long sum = 0;
    for (int i = 0; i < NUM_SAMPLES; ++i)
    {
        sum += analogRead(BATTERY_ADC_PIN);
    }
    float avg_adc = (float)sum / (float)NUM_SAMPLES;
    return avg_adc * VOLTAGE_STEP;
}
```

Figure 4.7: Code of Battery Monitoring

The ADC values are averaged multiple samples to reduce noise and improve stability.

4.3.7. Physical Arrangement of the System

The system components are arranged to simulate real-world conditions:

- Sensor node and control unit are placed in close proximity to minimize loss.
- Clear line-of-sight is maintained for wireless communication.
- DHT11 sensor is placed above plant level and soil moisture sensor is buried partially under soil for accurate measurements.
- Solar panel is installed at a suitable angle for sunlight exposure.
- Battery and electronics are placed outside the model.

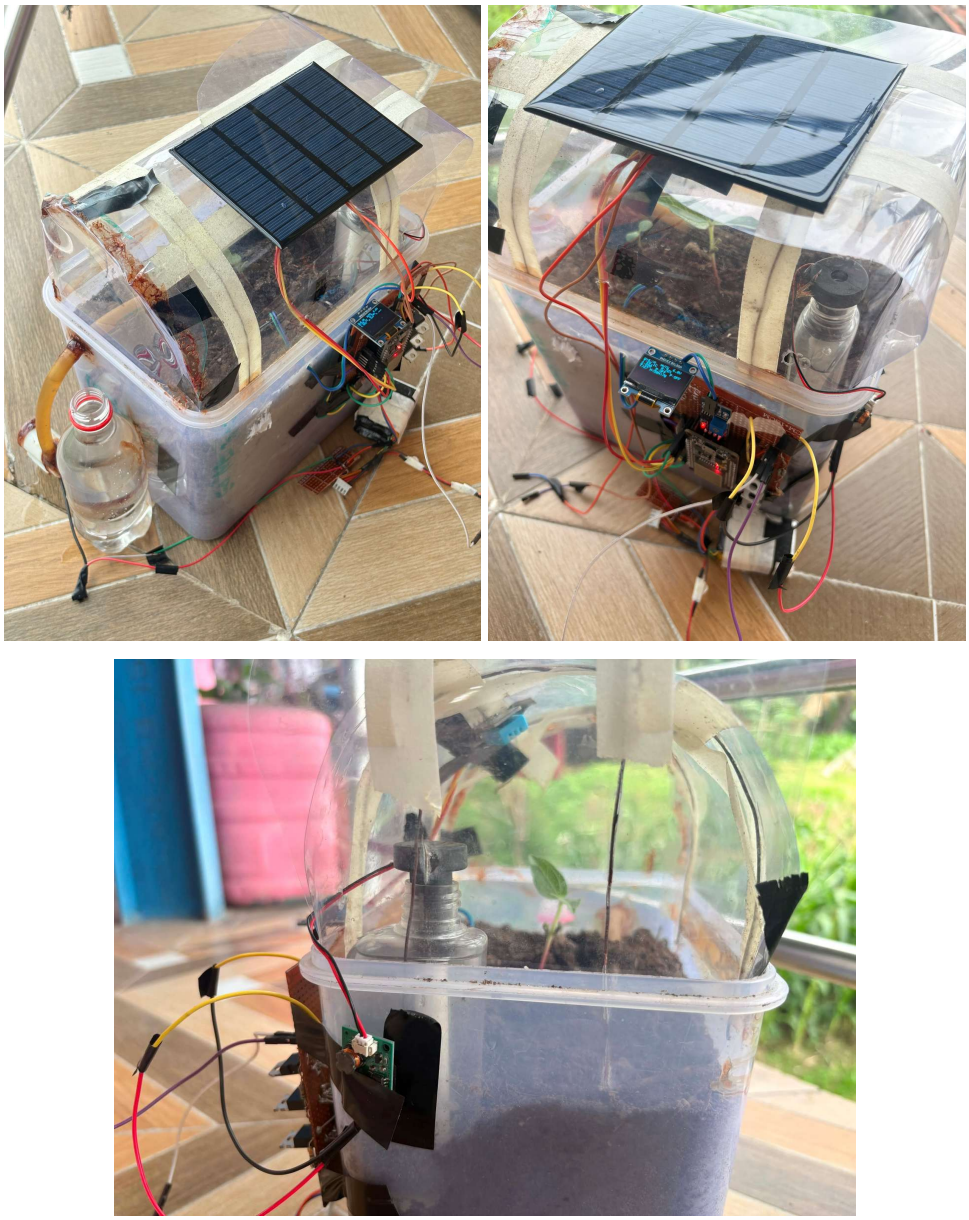


Figure 4.8: Physical Arrangement of Greenhouse

Greenhouse Physical Parameters (Mimic Model)

- Length = 20 cm
- Breadth = 14 cm
- Soil Height = 15 cm
- Height above soil = 13 cm
- For semi circular top:

  - Diameter = 14 cm
  - Height = 13 cm
- Total height = 28 cm

Geometrical and Physical Analysis

Area of base :

$$A = 20 \times 14 = 280\,\text{cm}^2$$

Volume of soil:

$$V_{soil} = 20 \times 14 \times 15 = 4200\,\text{cm}^3$$

Air volume:

$$V_{air} = 20 \times 14 \times 13 = 3640\,\text{cm}^3$$

The semi-circular top structure helps improve humidity retention and mimics real greenhouse airflow conditions.

4.3.8. Pin Configuration and Peripheral Interface

The ESP32 microcontroller as the central processing unit of the system interfacing all sensors, actuators and display modules. The system uses GPIO pins , ADC pins and I2C interfaces for hardware communication.

The OLED display is connected using the I2C protocol which has Serial Data and Serial Clock lines for synchronous communication. The display operates at the I2C address 0x3C, which is used by the ESP32 for initialization and data transmission.

S.No	Component	ESP32 Pin	Pin Type	Signal Type
1	DHT11 Sensor	GPIO 5	Bidirectional	Digital (Input)
2	Soil Moisture Sensor	GPIO 34	Input	Analog (ADC)
3	Mist Maker	GPIO 18	Output	Digital (ON/OFF / PWM)
4	Water Pump	GPIO 19	Output	Digital (ON/OFF)
5	OLED SDA	GPIO 21	Bidirectional	I2C Data Line
6	OLED SCL	GPIO 22	Output (Clock)	I2C Clock Line

Table 4.2: ESP32 Pin Configuration and Peripheral Interface

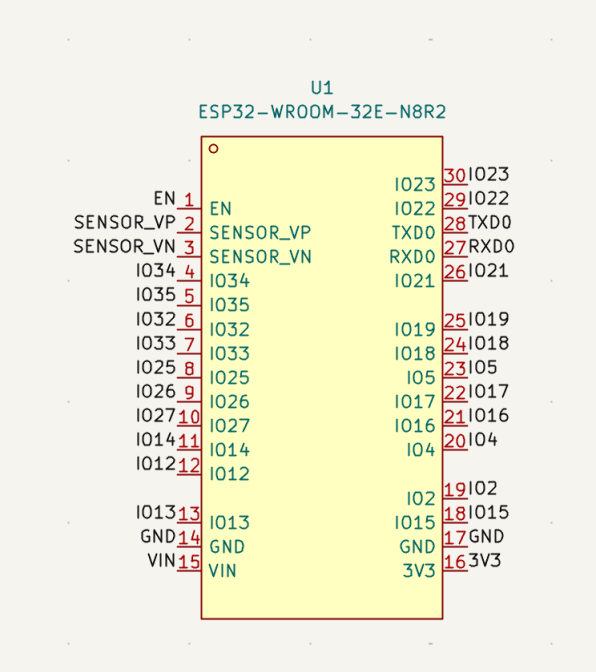


Figure 4.9: ESP32 Pin Configuration

4.3.9. Data Representation

The system data is displayed locally on an OLED interface, showing:

- System mode (AUTO/MAN)
- Temperature (°C)
- Humidity (%)
- Soil moisture (0–4095)
- Mist maker status (ON/OFF)
- Pump status (ON/OFF)
- Threshold values
- Battery voltage(Volts)

This enables local monitoring without external devices.

4.4. Automation Logic and Threshold Calibration

The system uses a threshold based control strategy to automate environmental regulation inside the greenhouse. The control logic continuously compares real-time sensor readings with predefined limits and activates or deactivates actuators accordingly.

- The water pump is activated when soil moisture falls below the lower threshold.
- Mist maker is activated when temperature exceeds the upper threshold or humidity falls below the desired range.
- Sensor values are updated at fixed intervals using the ESP32 microcontroller.

The threshold values are selected based on agronomic requirements for black-eyed pea (cowpea, *Vigna unguiculata*), which grows optimally under controlled temperature and moisture conditions.

Based on these conditions, the system uses:

- Soil moisture threshold: 2200 (Pump ON below this level)
- Humidity threshold: 50% (Mist ON below this level)
- Temperature threshold: 27°C (Mist ON above this level)

The control strategy uses hysteresis i.e, actuators switch OFF only when values return to the normal range, preventing frequent switching and improving system stability.

Actuator Control Implementation

```
// Actuator control logic
bool mistOn = false;
bool pumpOn = false;

if (autoMode)
{
    bool mistShouldBeOn = (temperature > temp_high_threshold) || (humidity < humidity_low_threshold);
    if (humidity > humidity_high_threshold) {
        mistShouldBeOn = false;
    }

    if (mistShouldBeOn)
    {
        digitalWrite(MIST_PIN, HIGH);
        mistOn = true;
    }
    else
    {
        digitalWrite(MIST_PIN, LOW);
        mistOn = false;
    }

    if (soilMoistureValue > soil_threshold)
    {
        digitalWrite(PUMP_PIN, HIGH);
        pumpOn = true;
    }
    else
    {
        digitalWrite(PUMP_PIN, LOW);
        pumpOn = false;
    }
}
else
{
    if (mistManualOn)
    {
        digitalWrite(MIST_PIN, HIGH);
        mistOn = true;
    }
    else
    {
        digitalWrite(MIST_PIN, LOW);
        mistOn = false;
    }

    if (pumpManualOn)
    {
        digitalWrite(PUMP_PIN, HIGH);
        pumpOn = true;
    }
    else
    {
        digitalWrite(PUMP_PIN, LOW);
        pumpOn = false;
    }
}
```

Figure 4.10: Code of ESP32 Actuator Control Logic

```
1    float temp_high_threshold = 27.0;
2    float temp_low_threshold = 21.0;
3    float humidity_low_threshold = 50.0;
4    float humidity_high_threshold = 70.0;
5    int soil_threshold = 2200;
```

Figure 4.11: Code of Threshold Values

4.5. Sensor Data Filtering and Noise Reduction

Raw sensor readings obtained from the DHT11 humidity and temperature sensor are often affected by transient fluctuations, electrical noise, environmental disturbances, and sensor instability. These fluctuations can lead to inaccurate environmental monitoring and unstable actuator triggering within the greenhouse automation system.

To improve measurement stability and enhance the reliability of automation decisions, a moving average filtering technique was implemented in the ESP32 firmware. The filtering algorithm continuously computes the average of the most recent sensor samples using a fixed-size sliding window.

The moving average equation is given by:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i \tag{4.1}$$

where:

- $\bar{x}$ = averaged sensor value
- N = total number of samples
- x_i = individual sensor readings

The above algorithm stores temperature and humidity readings inside circular buffer arrays and maintains cumulative sums for averaging. Instead of recalculating the complete summation during every update cycle, the oldest sample is subtracted and the newest sample is added. This significantly reduces computational overhead and improves processing efficiency. The computational complexity of the implemented filtering approach becomes:

$$O(1) \tag{4.2}$$

instead of:

$$O(N) \tag{4.3}$$

which is highly suitable for low-power embedded systems such as the ESP32.

The filtering process improves:

- Sensor stability

- Noise suppression
- Actuator response consistency
- Environmental monitoring accuracy

The invalid sensor readings were rejected before processing. The firmware checks for undefined numerical values using `isnan()` validation to prevent corrupted sensor data from affecting automation logic.

The implemented moving average filtering algorithm is shown below:

```
bool updateDhtAverages(float temperature, float humidity,
                       float &avgTemperature,
                       float &avgHumidity)
{
    if (isnan(temperature) || isnan(humidity))
    {
        return false;
    }

    if (dhtSampleCount < DHT_AVG_SAMPLES)
    {
        dhtTempSamples[dhtSampleIndex] = temperature;
        dhtHumiditySamples[dhtSampleIndex] = humidity;

        dhtTempSum += temperature;
        dhtHumiditySum += humidity;

        dhtSampleCount++;
    }
    else
    {
        dhtTempSum -= dhtTempSamples[dhtSampleIndex];
        dhtHumiditySum -= dhtHumiditySamples[dhtSampleIndex];

        dhtTempSamples[dhtSampleIndex] = temperature;
        dhtHumiditySamples[dhtSampleIndex] = humidity;

        dhtTempSum += temperature;
        dhtHumiditySum += humidity;
    }

    dhtSampleIndex =
        (dhtSampleIndex + 1) % DHT_AVG_SAMPLES;

    avgTemperature =
        dhtTempSum / (float)dhtSampleCount;

    avgHumidity =
        dhtHumiditySum / (float)dhtSampleCount;

    return true;
}
```

Figure 4.12: Code of DHT11 Moving Average Filtering Algorithm

The filtering mechanism significantly reduced random fluctuations in sensor readings and improved the overall stability of the greenhouse automation system during practical operation.

4.6. BLE-Based Monitoring and Data Management System

The system uses Bluetooth Low Energy (BLE) communication protocol between the ESP32 microcontroller and the user local monitoring device. BLE enables low power and short range wireless transmission of data without the internet.

The ESP32 operates as a BLE device and server while the remote monitoring system acts as a BLE client. This architecture ensures efficient and low latency data exchange that is suitable for a small-scale greenhouse system.

4.6.1. BLE Communication Architecture

The BLE communication framework is designed to support bidirectional data exchange between the greenhouse system and the monitoring interface.

The ESP32 server is responsible for continuously transmitting processed sensor readings and actuator status updates. The client device receives this data and presents it through a user interface for real-time monitoring.

In addition to data transmission, the system supports MANUAL mode optionally, where the client can send control instructions to change actuator states.

Functionality:

- Transmits real-time sensor data
- Provides actuator status updates
- Optionally receives user commands if in Manual mode

4.6.2. User Monitoring Interface

The data received through BLE communication is presented on a user monitoring interface, enabling real-time monitoring and visualization of greenhouse environmental conditions. This interface acts as the visualization layer of the system, translating transmitted sensor data into readable values for user interpretation.

The interface displays key environmental and system parameters received from the ESP32, allowing continuous observation of greenhouse conditions.

Monitored Parameters

- Temperature and humidity values
- Soil moisture level

- Water pump status
- Mist maker status
- Battery level

The interface may include basic visual indicators such as graphical representations to improve readability. It supports both mobile and desktop based applications depending on the deployment environment.

4.6.3. Data Handling and Logging

The system handles data at the client side after receiving structured sensor readings from the ESP32 via BLE communication. The received data is parsed and used for real-time monitoring of greenhouse parameters such as temperature, humidity, soil moisture, and actuator status.

The client device may also store the received data locally, enabling basic logging for future reference and trend analysis.

4.6.4. System Data Flow Architecture

- Sensor nodes collect environmental data.
- Data is transmitted to the ESP32 control unit for processing.
- Automation logic is applied based on thresholds.
- Processed data is transmitted via BLE.
- The user device receives and displays the data.

4.6.5. Monitoring Capability

The system supports short-range monitoring within BLE communication range (theoretically 100m). It enables real-time remote data access with low power consumption and reliable wireless communication without requiring internet.This makes the system suitable for small-scale greenhouse automation applications.

Although the theoretical BLE communication range of the ESP32 is approximately 100 m under ideal conditions, practical testing showed an effective range of about 50-60 m under near line of sight conditions. This reduction is due to environmental interference, signal attenuation, and real world deployment constraints.

4.6.6. Performance Analysis

The BLE communication performance of the system was evaluated using the nRF Connect mobile application. The ESP32 was successfully detected and operated using the configured service UUID `FFF0`, confirming correct implementation of the GATT profile.

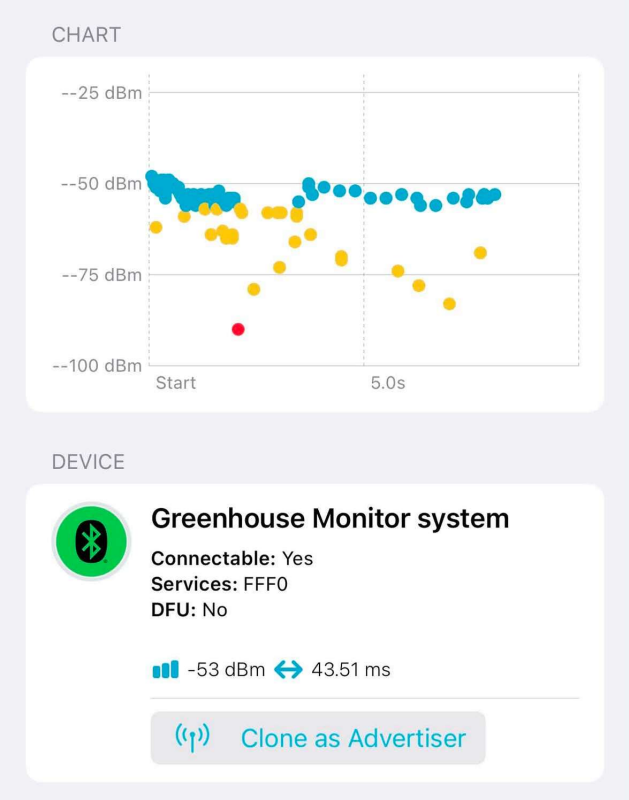


Figure 4.13: RSSI Chart

During testing, the system showed an average RSSI value of approximately -53 dbm, with variations ranging between -47 dBm and -56 dBm across different received packets. The measured connection interval of 43.51 ms indicates low-latency communication, which is suitable for real-time greenhouse monitoring applications.

The BLE communication system demonstrated stable connectivity, acceptable latency, and reliable data transmission performance. This ensures effective wireless monitoring and control within the intended operational range of the greenhouse system.

4.7. Software System Design

The greenhouse automation system follows a layered architecture consisting of ESP32 firmware, BLE communication, a Node.js gateway, a NestJS backend with PostgreSQL database, and a Next.js dashboard. The system is designed for efficient sensing, communication, processing, and real-time monitoring and control.

4.7.1. System Architecture and Operation

The ESP32 collects environmental data such as temperature, humidity, and soil moisture as well as controls actuators like the mist maker i.e humidifer and water pump. Data is transmitted to a Node.js gateway using Bluetooth Low Energy (BLE) communication protocal that forwards to the backend server.

The backend system developed using NestJS and PostgreSQL processes, validates and stores sensor data while managing system logic and command execution. The frontend dashboard built with Next.js provides monitoring, graphs analysis and auto or manual control capabilities.

The complete system is organized into five layers:

- **ESP32 Firmware Layer:** Sensor reading and actuator control
- **BLE Communication Layer:** Wireless data transmission
- **Gateway Layer (Node.js):** Data forwarding and command delivery
- **Backend Layer (NestJS + PostgreSQL):** Data processing and storage
- **Frontend Layer (Next.js Dashboard):** Monitoring and user interaction

4.7.2. Firmware and BLE Communication

Firmware (ESP32)

The firmware runs on the ESP32 and is responsible for sensor reading, actuator control and BLE communication. It continuously collects data from sensors such as temperature, humidity and soil moisture while filtering invalid readings to improve measurement accuracy.

In AUTO mode, it controls actuators according to pre-defined threshold values of the sensors. In MANUAL mode, it executes commands received from the dashboard to backend through BLE communication. The firmware sends sensor data to the gateway and updates system status information on the OLED display.

BLE Interface

The system implements Bluetooth Low Energy (BLE) using a custom GATT service on the ESP32 that enables communication with the gateway. Sensor data such as temperature, humidity, soil moisture and battery voltage are transmitted using read-/notify characteristics while control parameters such as operating mode, actuator states and threshold values are managed through read/write characteristics.

This architecture supports bidirectional communication allowing both monitoring , remote actuator control and setting the threshold values. The ESP32 operates as a BLE server while the gateway acts as a BLE client that subscribes to sensor updates and sends commands when required.

```
// BLE UUIDs
#define SERVICE_UUID "FFF0"

#define SENSOR_CHAR_UUID "FFF1"
// "temp,humid,soil,batteryV"
// Example: "24.5,60.0,2100,7.8"

#define CTRL_CHAR_UUID "FFF4"
// "mode,mist,pump"
// Example: "0,1,0"

#define THRESH_CHAR_UUID "FFF7"
// "tempThresh,soilThresh"
// Example: "25.0,2500"
```

Figure 4.14: Code of BLE UUID Configuration for ESP32 Communication

The configuration enables bidirectional communication between the ESP32 and gateway for greenhouse monitoring and control.

4.7.3. Backend, Frontend, and System Flow

Backend System

The backend system is developed using NestJS with TypeScript and PostgreSQL. It receives sensor data from the BLE gateway and validates incoming values, processes the information and stores it in the database for both real-time monitoring and historical analysis. The backend manages actuator commands, generates alerts, and exposes REST APIs for communication with both the frontend dashboard and the BLE gateway.

To improve system safety and reliability, the backend applies runtime limits, cooldown periods, command expiration handling, and alert deduplication mechanisms.

Frontend System

The frontend dashboard is built using Next.js with TypeScript. It provides real-time monitoring of environmental conditions and actuator status through a user-friendly interface.

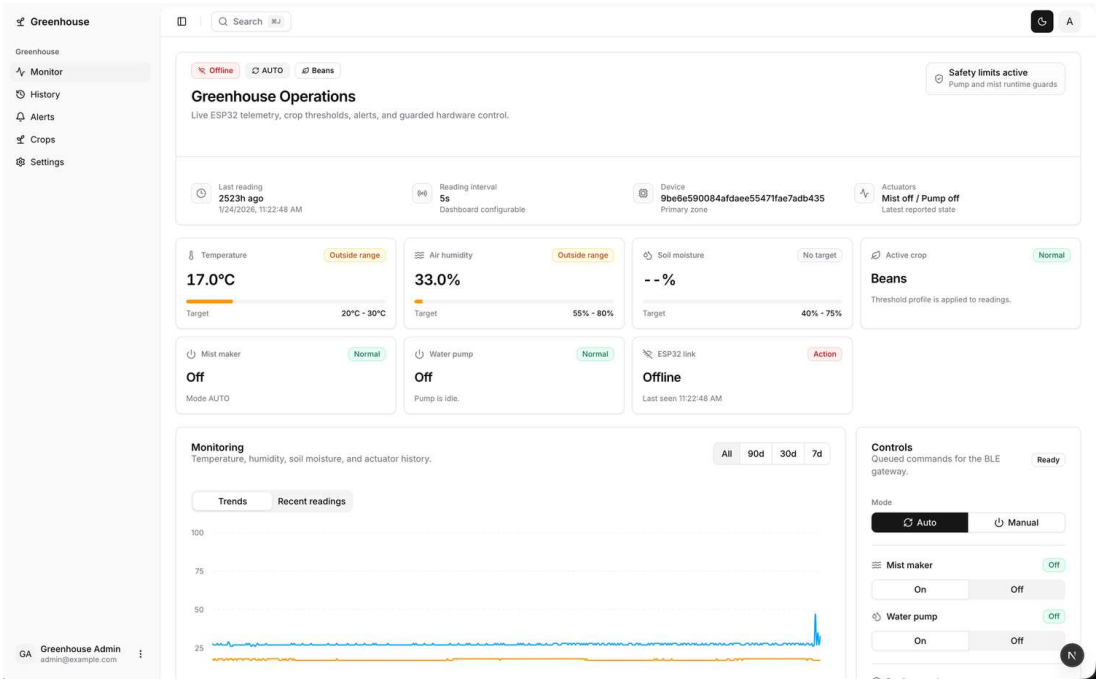


Figure 4.15: Frontend Dashboard

The dashboard displays:

- Temperature and humidity readings
- Soil moisture percentage
- Water pump status
- Mist generator status
- Alert notifications

The frontend periodically fetches updated data from the backend using REST APIs to maintain near real-time monitoring. Authentication and role-based access control are also implemented to ensure secure access to system control functions.

Data Flow Architecture

The system implements bidirectional data flow for monitoring and control operations.

Sensor Data Flow

- ESP32 reads environmental sensor data
- Data is transmitted to the gateway via BLE
- Gateway forwards data to the backend server
- Backend validates and stores data in PostgreSQL
- Dashboard fetches and displays data using REST APIs

Control Command Flow

- User actions from the dashboard are sent to the backend
- Backend creates and manages control commands
- Gateway fetches pending commands from the backend
- Commands are transmitted to ESP32 through BLE write operations
- ESP32 executes commands and updates actuator states

4.7.4. Database, API and Deployment

The system uses PostgreSQL with Prisma ORM to manage storage of sensor readings, user accounts, actuator states, commands and alert records.

The backend exposes REST APIs for:

- Sensor data ingestion
- Data retrieval and visualization
- User authentication
- Actuator control and command management

The complete greenhouse automation system is deployed as independent functional components:

- ESP32 for sensing and actuator control
- BLE protocal for wireless communication

- Backend server for processing and management
- PostgreSQL database for persistent storage
- Web dashboard for monitoring and user interaction

Together, these integrated modules provide a reliable and scalable solar-powered greenhouse automation system capable of real-time monitoring, automated environmental control, and remote management.

5. EPILOGUE

5.1. Results and Findings

A real time greenhouse automation system was successfully implemented using the ESP32 module, DHT11 and soil moisture sensors, and threshold based automation control. The system continuously monitored temperature, humidity and soil moisture, and activated the actuators to balance the internal parameters to maintain a stable greenhouse environment. Realtime analysis and automatic control of irrigation and humidifier , reduced the manual effort and optimized the water usage.

5.1.1. Relation in Auto and Manual Mode

The results and graphs from our experimental data are as below:

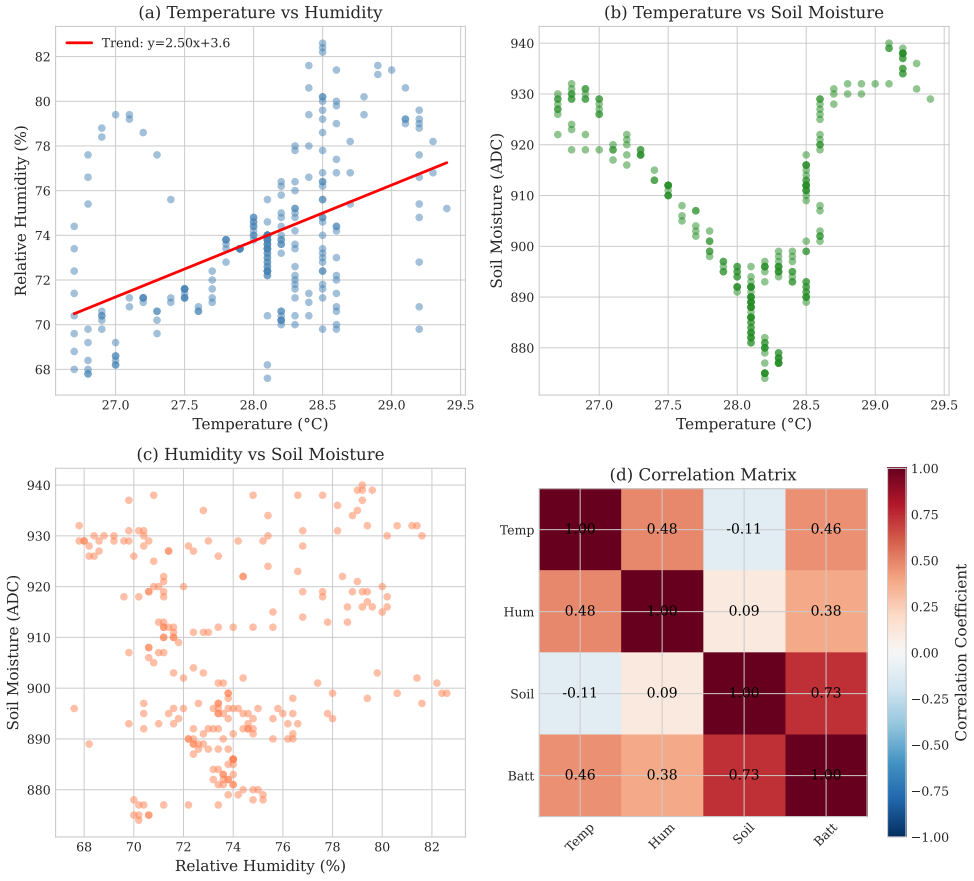


Figure 5.1: Correlation analysis in AUTO Mode

The graphs visualize the data while greenhouse system was operating in AUTO mode.

Graph a. shows that humidity increases when temperature increases.
Graph b. shows that soil moisture changes with temperature.
Graph c. shows that humidity and soil moisture are regulated automatically.
The Correlation Matrix indicates the system maintains an environmental balance.

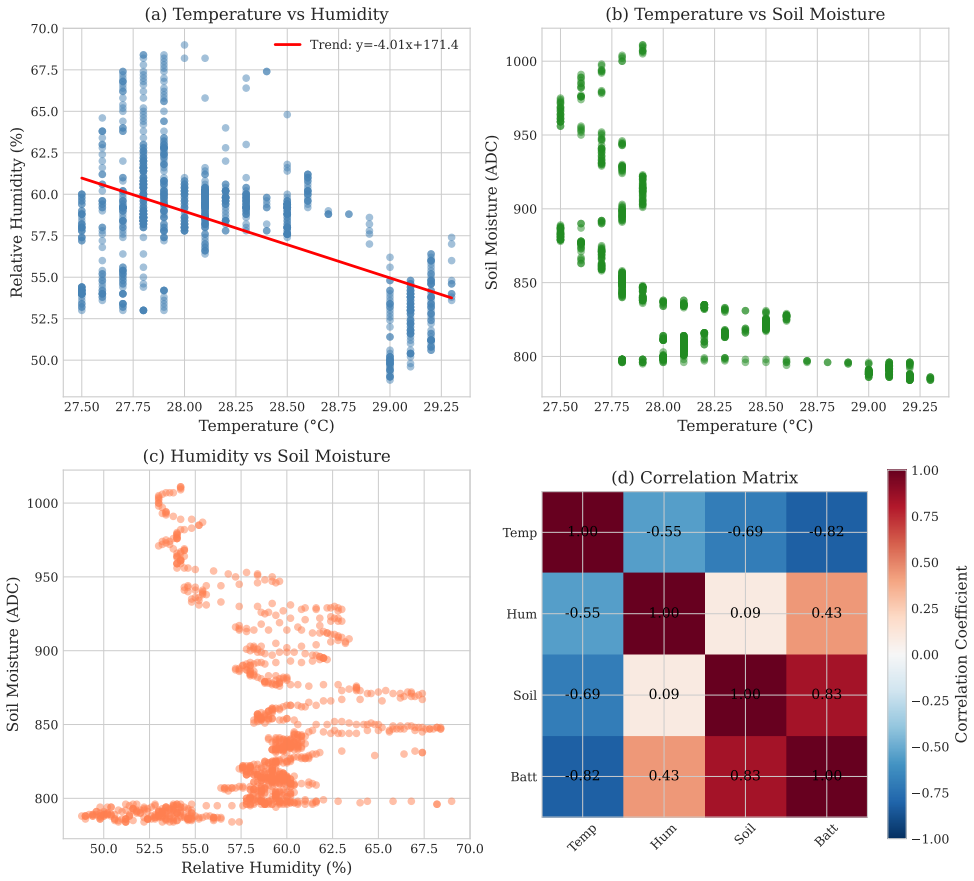


Figure 5.2: Correlation analysis in MANUAL Mode

The graphs represents the greenhouse system data when operating in MANUAL mode.

Graph a. shows temperature rises when humidity decreases.
Graph b. shows variations in soil moisture as temperature changes.
Graph c. shows that soil irrigation is not well maintained.
The Correlation Matrix shows environmental conditions being imbalanced during manual operation.

Table 5.1 compares the system performance in AUTO and MANUAL operating modes.

Parameter	AUTO Mode	MANUAL Mode	Operating Threshold
Number of Samples	835	1056	–
Temperature (°C)	27.32 ± 0.76	28.17 ± 0.52	21°C – 27°C
Humidity (%)	71.9 ± 3.5	58.1 ± 3.8	50% – 70%
Soil Moisture (ADC)	880 ± 31	854 ± 72	< 2200
Battery Voltage (V)	6.47 ± 0.35	7.19 ± 0.13	–
Mist Maker Activation (%)	11.5%	0.7%	–

Table 5.1: Performance Comparison Between AUTO and MANUAL Operating Modes

The table shows that AUTO mode is more stable than the MANUAL mode.

5.1.2. Environmental Time Series

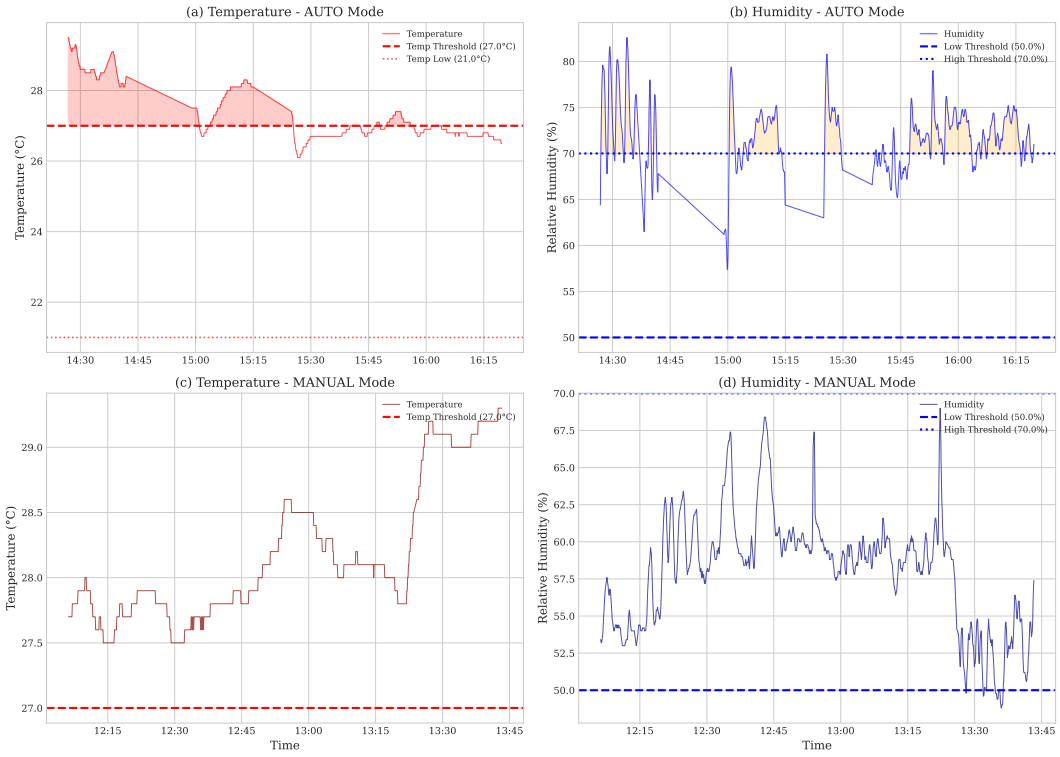


Figure 5.3: Environmental Time Series

The graphs show the variation in greenhouse parameters over time in AUTO and MANUAL modes.

In AUTO mode, temperature and humidity lie within a stable range near the thresholds.

In MANUAL mode, temperature have irregular spikes and humidity goes high due to manual delays.

The AUTO mode has stable environmental behavior as compared to MANUAL mode.

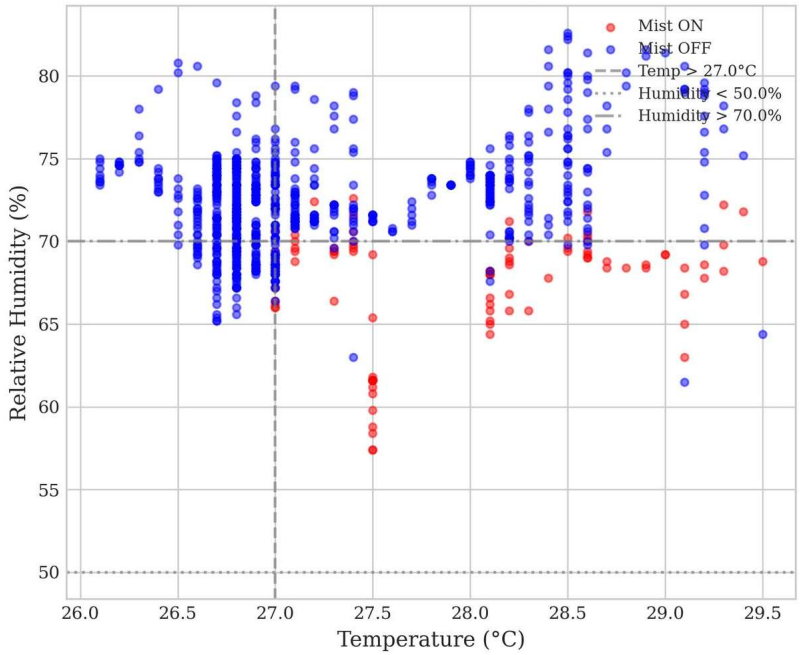


Figure 5.4: Activation Regions of Mist Maker

The graph shows the temperature and humidity values at which the mist maker is actually turned ON/OFF during the testing. It is observed that the mist maker is continuously activated when the temperature and humidity are not in desired range.

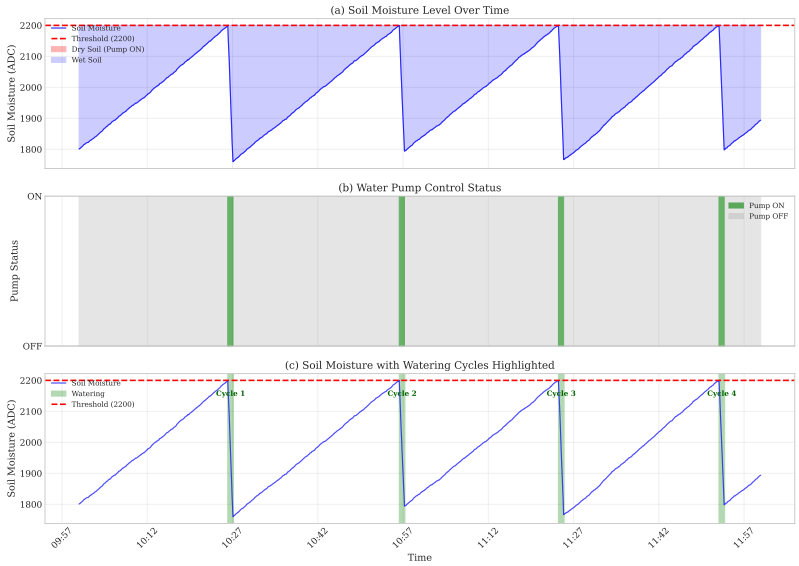


Figure 5.5: Motor Pump ON/OFF

The figure shows the soil moisture control situation in AUTO mode. It displays the soil moisture level over time, the pump ON/OFF state and irrigation cycles. It demonstrates that the soil moisture level were maintained well even without manual interference.

5.1.3. Solar Charging and Energy Performance Analysis

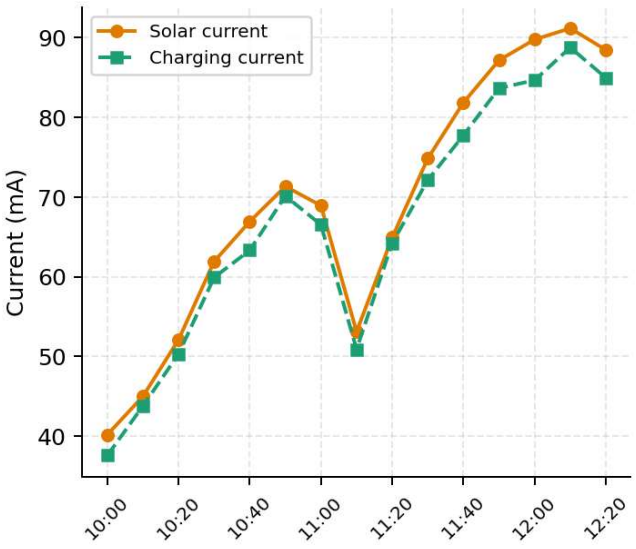


Figure 5.6: Solar Current vs Charging Current

The graph shows variation in solar charging battery and current during different times of a day. The fluctuations are seen due to clouds.

The solar current reaches upto 91 mA and the charging current was seen upto 89 mA.

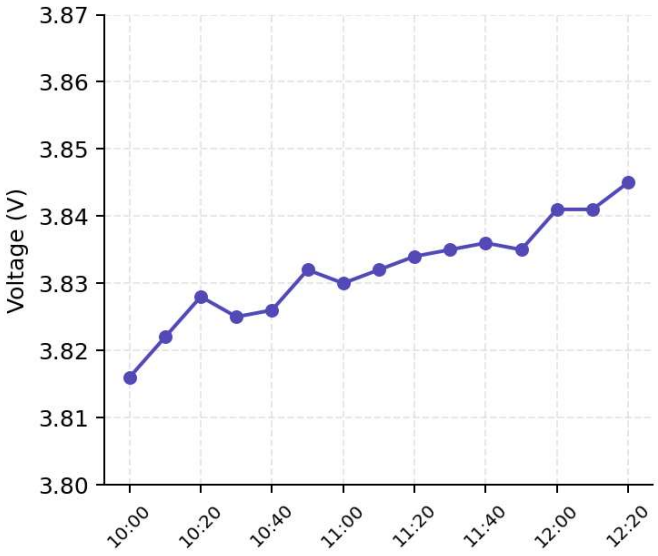


Figure 5.7: Battery Voltage Variation

The battery voltage increased from 3.816 V to 3.845 V during the observation period.

The increasing voltage means the solar charging system is working.

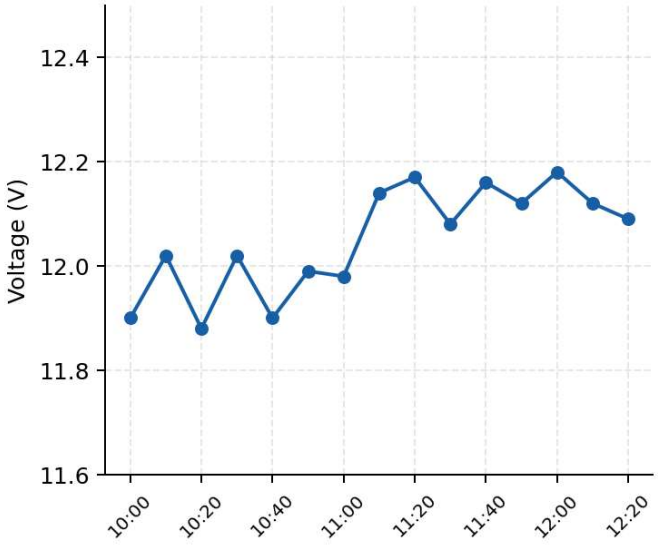


Figure 5.8: Solar Panel Voltage Variation

The solar panel voltage was between 11.9 V and 12.2 V.

5.1.4. Key Findings

- The system performs greenhouse automation.
- AUTO mode is more stable than MANUAL mode.
- Battery voltage fluctuates when the actuators are switching.
- System runtime=15 hours, and power consumption=0.64 W.
- Solar charging efficiency varies with sunlight and clouds.
- BLE communication was found reliable for a range of 50 - 60m line of sight.

5.2. Limitations and Future Enhancements

5.2.1. Limitations

- Multiple battery charging.
- Slow charging
- Low scale application only
- Range of BLE is less
- Single point of failure

5.2.2. Future Enhancements

- Using multicell BMS
- Upgrade the solar panel
- ML and AI Integration and analysis
- Using multiple BLE nodes/LoRa/Wi-fi for long distance communication

5.3. Conclusion

In this project, we developed a solar based greenhouse automation system that can monitor and control the internal environment of a greenhouse without continuous human supervision. It monitors temperature, humidity and soil moisture and automatically activates the actuators. The water pump is triggered when the soil becomes dry and the mist maker is triggered when either humidity drops or temperature rises above the set threshold, helping maintain suitable growing conditions for cowpea plant. The system is controlled by an ESP32 microcontroller which reads values from a DHT11 sensor and soil moisture sensor, then controls water pump and mist maker, and displays on an OLED display for real time readings.The data is also transmitted using BLE to a web dashboard for remote monitoring and control. A series of LiPo batteries power the system and can be charged by solar allowing it to operate without grid electricity. While testing, the system performed better in AUTO mode, maintaining more stable environmental conditions compared to manual control. Irrigation and misting responded accurately to sensor inputs, and overall power consumption remained low. The solar system was practical for off-grid use. Although the charging current of around 91 mA resulted in slower charging even under direct sunlight. Despite that, a fully charged battery supported about 15 hours of operation at 0.64 W. A moving average filter was implemented to stability by reducing noise in ADC readings, which prevented false actuator triggers. BLE provided stable communication till almost 50–60 m. The system provides a simple, practical, and low-cost solar-based greenhouse automation solution for farming in rural Nepal, where access to electricity and advanced systems is limited.

References

[1] S. Candy, G. Moore, and P. Freere, "Design and modeling of a greenhouse for a remote region in nepal," *Procedia Engineering*, vol. 49, pp. 152–160, 2012.

[2] M. F. Siddiqui, A. u. R. Khan, N. Kanwal, H. Mehdi, A. Noor, and M. A. Khan, "Automation and monitoring of greenhouse," in *2017 International Conference on Information and Communication Technologies (ICICT)*, 2017, pp. 197–201.

[3] F. Abou-Mehdi-Hassani, A. Zaguia, H. Ait Bouh, and A. Mkhida, "Systematic literature review of smart greenhouse monitoring," *SN Computer Science*, vol. 6, no. 2, p. 95, 2025.

[4] M. Omid and A. Shafaei, "Temperature and relative humidity changes inside greenhouse," *International Agrophysics*, vol. 19, no. 2, pp. 153–158, 2005.

[5] A. Ayuba, "Dht11 based temperature and humidity measuring system," https://www.researchgate.net/publication/376679012_Dht11_Based_Temperature_And_Humidity_Measuring_System, 2023, accessed: 2026-05-01.

[6] K. K. Singh, "Solar energy – the future of energy," pp. 881–884, 2024, accessed: 2026-05-01. [Online]. Available: https://www.ijtsrd.com/papers/ijtsrd67224.pdf

[7] Wikipedia, "Thin-film solar cell," 2026, accessed: 2026-05-01. [Online]. Available: https://en.wikipedia.org/wiki/Thin-film_solar_cell

[8] Lion Care, "Lipo batteries: Properties, advantages and more," 2026, accessed: 2026-05-01. [Online]. Available: http://lion-care.com/en/Info-Center/Industry-knowledge/Overview-of-important-battery-types-and-technologies/LiPo-batteries-properties-advantages-and-more/

[9] Random Nerd Tutorials, "Getting started with the esp32 development board," 2016, accessed: 2026-05-01. [Online]. Available: https://randomnerdtutorials.com/getting-started-with-esp32/

[10] Mouser Electronics, "Dht11 technical data sheet," 2026, accessed: 2026-05-01. [Online]. Available: https://www.mouser.com/datasheet/2/758/DHT11-Technical-Data-Sheet-Translated-Version-1143054.pdf

[11] HandsOnTec, “Soil moisture sensor datasheet,” 2026, accessed: 2026-05-01. [Online]. Available: https://handsontec.com/dataspecs/sensor/Soil%20Moisture%20Sensor.pdf

[12] Ingco, “What is a water pump,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.ingco.com/blog/what-is-water-pump

[13] Piezo Ultrasonic, “Ultrasonic mist fogger a015,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.piezo-ultrasonic.com/ultrasonic-mist-fogger-a015.html

[14] OLED-Info, “Oled introduction,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.oled-info.com/oled-introduction

[15] Scribd, “Tp4056 english documentation,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.scribd.com/document/117369667/TP4056-English-Documentation

[16] Monolithic Power Systems, “Mp1584en datasheet,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.monolithicpower.com

[17] Infineon Technologies, “Irlz44n mosfet datasheet,” 2026, accessed: 2026-05-01. [Online]. Available: https://www.infineon.com

[18] PlatformIO, “What is platformio?” 2025, accessed: 2026-05-01. [Online]. Available: https://docs.platformio.org/en/latest/what-is-platformio.html

[19] KiCad Project, “Introduction to kicad,” 2022, accessed: 2026-05-01. [Online]. Available: https://docs.kicad.org

[20] NovelBits, “Bluetooth low energy (ble) complete guide,” 2026, accessed: 2026-05-01. [Online]. Available: https://novelbits.io/bluetooth-low-energy-ble-complete-guide/

[21] Last Minute Engineers, “Esp32 bluetooth classic tutorial and ble overview,” 2026, accessed: 2026-05-01. [Online]. Available: https://lastminuteengineers.com/esp32-bluetooth-classic-tutorial/

[22] M. S. Hossain, M. S. Islam, and M. R. I. Sheikh, “Basic design of solar charging circuit,” 2020, accessed: 2026-05-01. [Online]. Available: https://www.researchgate.net/publication/343258617